

199. 二叉树的右视图

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 广度优先搜索, 二叉树
- **Companies:** (Not Available)

给定一个二叉树的 **根节点** `root`，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例 1:

输入: `[1,2,3,null,5,null,4]`

输出: `[1,3,4]`

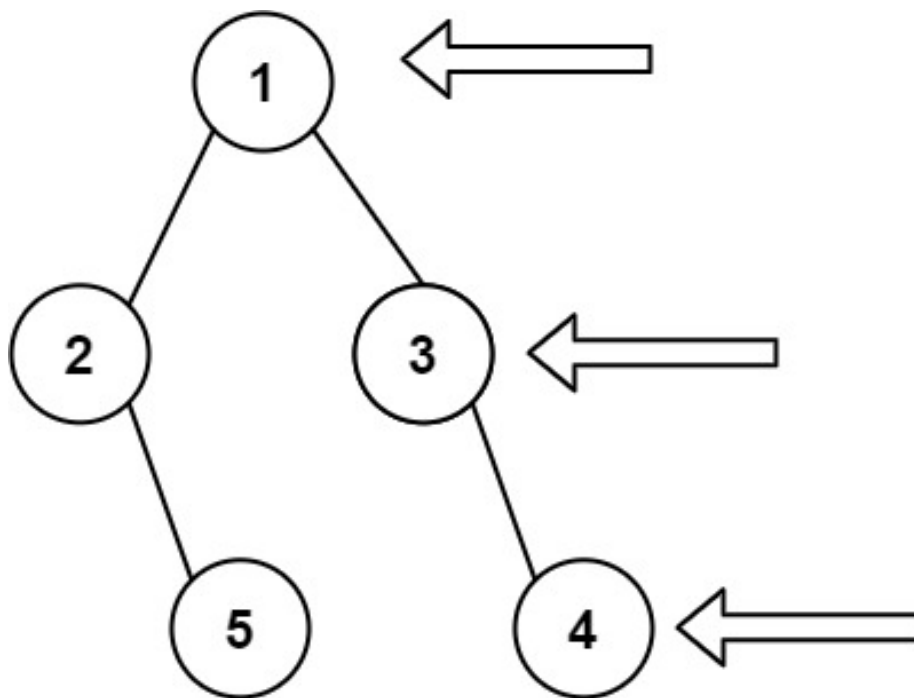


Figure 1: img

示例 2:

输入: `[1,null,3]`

输出: `[1,3]`

示例 3:

输入: []

输出: []

提示:

- 树中节点的数目在 [0, 100] 范围内
- $-100 \leq \text{Node.val} \leq 100$

Discussion | Solution

解决方法

1. 问题理解

目标: 返回二叉树的右视图, 即从右侧看过去能看到的节点值, 按从上到下的顺序排列。**关键:** 右视图看到的节点是每一层**最右边**的那个节点。

2. 思路分析

方法一: 广度优先搜索 (BFS / 层序遍历) 层序遍历天然地按层处理节点。我们可以在层序遍历的基础上, 只取每层的最后一个节点。

1. 初始化:

- 创建一个空列表 `result` 用于存储右视图节点值。
- 如果 `root` 为 `null`, 返回 `result`。
- 创建一个队列 `queue`, 并将根节点 `root` 入队。

2. 层序遍历循环:

- 当队列 `queue` 不为空时, 进行循环 (每次循环处理一层):
 - 获取当前层的节点数量 `level_size = len(queue)`。
 - **遍历当前层所有节点:** 循环 `i` 从 0 到 `level_size - 1`。
 - * 将队首节点 `node` 出队。
 - * **关键点:** 检查当前节点是否是该层的最后一个节点。如果是 (`i == level_size - 1`), 则将 `node.val` 加入 `result` 列表。
 - * 将非空的左右子节点入队 (顺序不影响结果, 但通常先左后右)。
 - `if node.left: queue.append(node.left)`
 - `if node.right: queue.append(node.right)`

3. 返回结果: 循环结束后, `result` 包含所有层最右侧节点的值, 返回 `result`。

执行流程示例 ([1,2,3,null,5,null,4]):

步骤	queue	level_size	i	node	result	操作
1	[1]	-	-	-	[]	初始化
2	[1]	1	-	-	[]	开始第 1 层

步骤	queue	level_size	i	node	result	操作
3	[]	1	0	1	[1]	出队 1, $i == \text{level_size} - 1$, 加入 result
4	[2, 3]	1	0	1	[1]	入队 2, 3
5	[2, 3]	2	-	-	[1]	开始第 2 层
6	[3]	2	0	2	[1]	出队 2, $i \neq \text{level_size} - 1$
7	[3, 5]	2	0	2	[1]	入队 5 (左子 null)
8	[5]	2	1	3	[1, 3]	出队 3, $i == \text{level_size} - 1$, 加入 result
9	[5, 4]	2	1	3	[1, 3]	入队 4 (左子 null)
10	[5, 4]	2	-	-	[1, 3]	开始第 3 层
11	[4]	2	0	5	[1, 3]	出队 5, $i \neq \text{level_size} - 1$ (无子)
12	[]	2	1	4	[1, 3, 4]	出队 4, $i == \text{level_size} - 1$, 加入 result (无子)
13	[]	-	-	-	[1, 3, 4]	队列为空, 结束, 返回 result

方法二：深度优先搜索 (DFS - 递归) DFS 也可以用来解决这个问题。关键在于记录当前遍历到的层级 (深度)，并确保对于每一层，我们只记录**第一个**访问到的节点 (如果我们优先遍历右子树) 或者**最后一个**访问到的节点 (如果我们优先遍历左子树，但需要覆盖)。

更巧妙的方法是利用 result 列表的长度来判断是否是第一次到达该层。

- 初始化：**
 - 创建一个空列表 result 用于存储右视图节点值。
- 定义递归函数 dfs(node, depth)：**
 - 功能：**访问 node 节点，depth 表示当前节点所在的层级 (根节点为 0)。
 - 基本情况 (Base Case)：**如果 node 为 null，返回。
 - 关键点：**检查当前深度 depth 是否等于 result 列表的长度。如果是，说明这是**第一次**访问到 depth 这一层 (因为我们优先访问右子树)，将 node.val 加入 result。
 - if depth == len(result): result.append(node.val)
 - 递归步骤 (Recursive Step) - 优先右子树：**为了确保首先看到的是右边的节点：
 - 先递归调用右子节点: dfs(node.right, depth + 1)。
 - 再递归调用左子节点: dfs(node.left, depth + 1)。
- 主函数调用：**
 - 调用 dfs(root, 0) 开始递归。
 - 返回 result。

执行流程示例 ([1,2,3,null,5,null,4]):

1. result = []
2. dfs(1, 0): depth(0) == len(result)(0), result = [1].
3. -> dfs(3, 1): depth(1) == len(result)(1), result = [1, 3].
4. -> dfs(4, 2): depth(2) == len(result)(2), result = [1, 3, 4].
5. -> dfs(null, 3) -> 返回
6. -> dfs(null, 3) -> 返回 (节点 4 左右子 null)
7. -> 返回 (节点 3 右子 4 处理完)
8. -> dfs(null, 2) -> 返回 (节点 3 左子 null)
9. -> 返回 (节点 3 处理完)
10. -> dfs(2, 1): depth(1) < len(result)(3), 不添加。
11. -> dfs(5, 2): depth(2) < len(result)(3), 不添加。
12. -> dfs(null, 3) -> 返回
13. -> dfs(null, 3) -> 返回 (节点 5 左右子 null)
14. -> 返回 (节点 2 右子 5 处理完)
15. -> dfs(null, 2) -> 返回 (节点 2 左子 null)
16. -> 返回 (节点 2 处理完)
17. 返回 (根节点 1 处理完)
18. 最终返回 result = [1, 3, 4]

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import List, Optional
import collections

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 广度优先搜索 (BFS)
    def rightSideViewBFS(self, root: Optional[TreeNode]) -> List[int]:
        """
        使用 BFS 获取二叉树的右视图
        """
        result = []
```

```

        if not root:
            return result

    queue = collections.deque([root])

    while queue:
        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            # 如果是当前层的最后一个节点, 加入结果
            if i == level_size - 1:
                result.append(node.val)
            # 将下一层的节点入队
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    return result

# 方法二: 深度优先搜索 (DFS - 递归)
def rightSideViewDFS(self, root: Optional[TreeNode]) -> List[int]:
    """
    使用 DFS 获取二叉树的右视图
    """
    result = []

    def dfs(node: Optional[TreeNode], depth: int):
        if not node:
            return

        # 如果当前深度等于 result 长度, 说明是该层第一个访问到的节点 (按 根-右-左 顺序)
        if depth == len(result):
            result.append(node.val)

        # 优先递归右子树, 保证先看到右边的
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return result

# 最终提交可以选择其中一种方法
def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
    # return self.rightSideViewBFS(root)
    return self.rightSideViewDFS(root)

```

4. 复杂度分析

- **方法一 (BFS):**
 - 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点入队出队各一次。
 - 空间复杂度: $O(W)$, 其中 W 是树的最大宽度。队列中最多存储一层节点, 最坏情况下 (完全二叉树的最后一层) 可能达到 $O(N)$ 。
- **方法二 (DFS):**
 - 时间复杂度: $O(N)$ 。每个节点被访问一次。
 - 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。result 列表的空间复杂度为 $O(H)$ 。

5. 如何在面试中讲解

1. **明确目标:**
 - “目标是找到二叉树的右视图, 也就是从右边看每一层能看到的第一个节点, 按从上到下的顺序组成列表。”
 - “这等价于找到每一层的最右边的那个节点。”
2. **方法一: BFS (层序遍历):**
 - “BFS 按层遍历, 非常适合这个问题。”
 - “在标准的层序遍历基础上, 记录每层的节点数 level_size。”
 - “在处理每一层时, 只将该层遍历到的最后一个节点的值加入结果列表。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(W)$ 。”
3. **方法二: DFS (递归):**
 - “DFS 也可以解决。关键在于记录当前节点的深度 depth。”
 - “为了模拟从右边看, 我们应该优先遍历右子树 (根 -> 右 -> 左的顺序)。”
 - “使用 result 列表存储结果。当递归到一个节点时, 检查它的深度 depth。如果 depth 等于 result 当前的长度, 说明这是我们第一次到达这一层 (因为优先访问右边), 就把这个节点的值加入 result。”
 - “如果 depth 小于 result 长度, 说明这一层已经有节点被记录过了 (肯定是更右边的节点), 就忽略当前节点。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”
4. **对比与选择:**
 - “BFS 思路更直观地对应“层”的概念。DFS 通过巧妙的遍历顺序和深度判断也能实现。”
 - “两者时间复杂度都是 $O(N)$ 。空间复杂度 BFS 取决于宽度 W , DFS 取决于高度 H 。”
5. **边界条件:**
 - “处理空树 (root 为 null) 的情况, 返回空列表。”